Experiment 8: Train a Neural Network to Classify Handwritten Digits (MNIST)

Aim / Objective

Write a program in python to implement a Neural Network to Classify Handwritten Digits (MNIST).

Theory / Introduction

Artificial Neural Networks (ANNs) are a set of algorithms inspired by the human brain, capable of learning complex relationships between inputs and outputs.

- **Input Layer**: Receives raw pixel data
- **Hidden Layers**: Learn intermediate features using activation functions (ReLU)
- **Output Layer**: Gives classification results using Softmax activation

This experiment uses a simple feedforward ANN (Multilayer Perceptron) to classify handwritten digits (0–9) from the MNIST dataset.

Software / Tools Required

- Python
- TensorFlow
- Keras
- Matplotlib

Dataset Used

MNIST Dataset

About the Dataset

The MNIST dataset contains **70,000 grayscale images** of handwritten digits (28x28 pixels) labeled from 0 to 9.

- 60,000 training images
- 10,000 testing images
- Each image is 28×28 pixels, flattened into a 784-dimensional vector

🔗 Easily available from Keras:

python

from tensorflow.keras.datasets import mnist

Procedure / Steps

1. Load the MNIST dataset
2. Normalize the pixel values
3. Build the ANN architecture
4. Train the model and evaluate accuracy
5. Visualize sample predictions

Python Step-by-Step Code with Explanations

```
# Step 1: Import necessary libraries
import numpy as np
import matplotlib.pyplot as plt
from tensorflow.keras.datasets import mnist
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Flatten
from tensorflow.keras.utils import to_categorical

# Step 2: Load dataset
(X_train, y_train), (X_test, y_test) = mnist.load_data()

# Step 3: Normalize data
X_train = X_train / 255.0
X_test = X_test / 255.0

# Step 4: One-hot encode labels
y_train = to_categorical(y_train, 10)
y_test = to_categorical(y_test, 10)

# Step 5: Build ANN model
model = Sequential()
```

```
model.add(Flatten(input_shape=(28, 28)))       # Flatten 28x28 to 784
model.add(Dense(128, activation='relu'))       # Hidden layer
model.add(Dense(10, activation='softmax'))     # Output layer for 10 classes

# Step 6: Compile and train the model
model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])
model.fit(X_train, y_train, epochs=5, batch_size=32, validation_split=0.1)

# Step 7: Evaluate on test set
loss, accuracy = model.evaluate(X_test, y_test)
print(f'Test Accuracy: {accuracy:.4f}')

# Step 8: Predict and visualize
predictions = model.predict(X_test)
plt.imshow(X_test[0], cmap='gray')
plt.title(f'Predicted Label: {np.argmax(predictions[0])}')
plt.show()
```

Result / Output

- Model was trained for 5 epochs using the MNIST dataset
- Achieved over 95% accuracy on the test data
- First sample image was visualized with its predicted label

Conclusion

Artificial Neural Networks effectively learn complex visual patterns from pixel data. The trained ANN model showed excellent performance in recognizing handwritten digits from the MNIST dataset.

Viva Questions (2–3 per experiment)

1. What is the role of the ReLU and Softmax activation functions?
2. Why is data normalization important in training neural networks?
3. What is the purpose of one-hot encoding for labels?